

EECS 498 · AASE · FALL 2026

# How Aider works

LAB01 · DESIGN YOUR PAIR-PROGRAMMER · SEP 14–15, 2026

# Schedule

1. Aider deep dive
2. Hackathon 1
3. The next assignment
4. Office hours

# The harness

Aider is a *harness*: a program that stands between you and a model.

| The harness                   | The model      |
|-------------------------------|----------------|
| Picks the text the model sees | Reads it       |
| Decides what a reply may do   | Writes a reply |
| Edits files, then commits     | Nothing        |

Take one apart now, build one later.

# What the model never touches

- Your disk
- Your shell
- Your last session

**Text in, text out. The harness does the rest.**

# What writes, and what does not

---

Arranges context

`/add`

`/read-only`

`/drop`

`/tokens`

`/ask`

`/diff`

---

Changes your files

`/code`

`/undo`

---

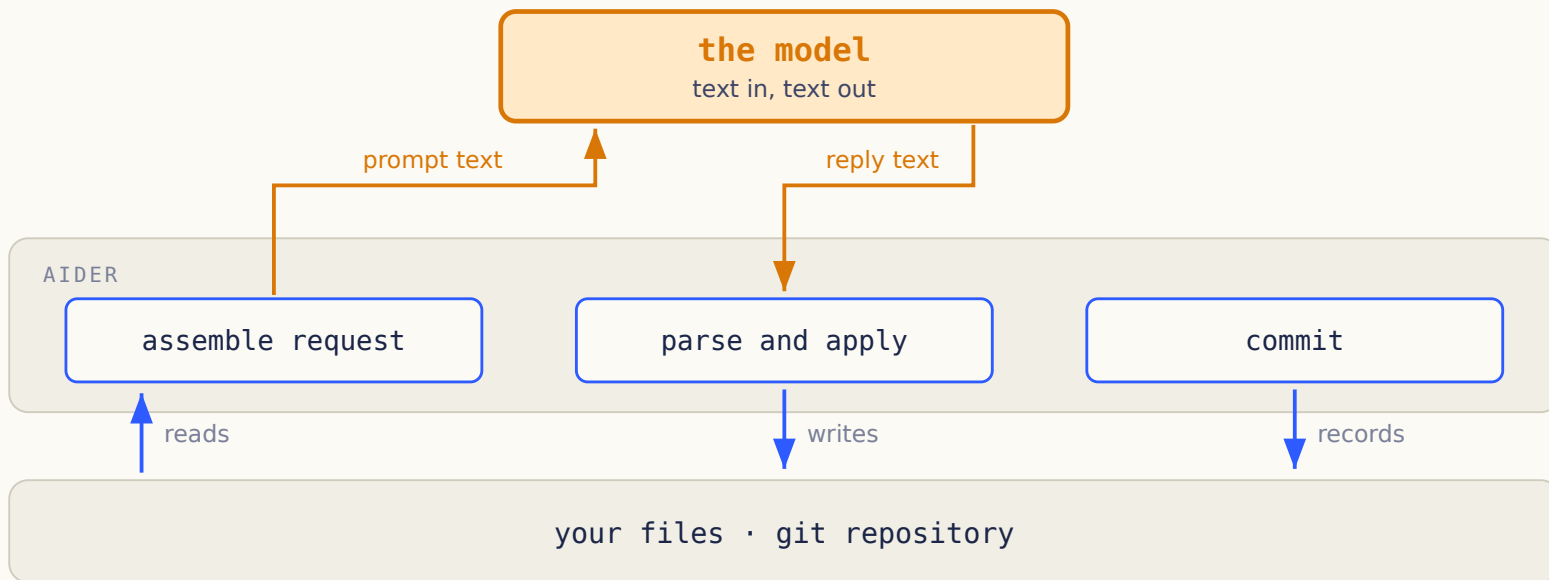
Bare text means `/code`. The default writes.

# Context

WHAT THE HARNESS CHOOSES TO SEND

WHO DOES WHAT

# Aider, the model, and the repo



The model touches no file.

# The repo map

```
package/store.py:
:
|class Store:
|    def add(self, title, tags=()) -> Task:
|    def remove(self, task_id: int) -> bool:
:
|    def search(self, text, tag=None) -> list[Task]:
:

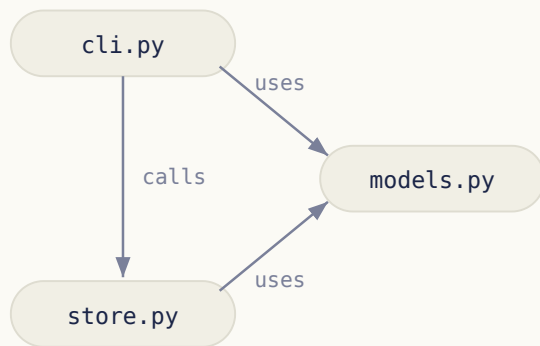
package/models.py:
|@dataclass
|class Task:
:
```

Signatures only. `:` is what was cut.

**Files you never added.**

# Where the repo map comes from

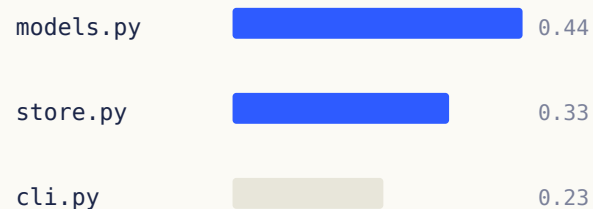
1. parse every file for what it defines and what it uses



models.py is used by both, so it wins

PageRank, rendered until the budget ends.

2. rank by what points at them



budget ends here

cli.py is left out entirely

# One shared token budget



One budget. Your message is what stops fitting.

**Your build decides what to drop. The packet does not.**

# Rebuilding the prompt

NOTHING CARRIES OVER, SO ALL OF IT IS SENT AGAIN

SORTED BY VOLATILITY

# The order of the prefix

| How often it changes | Which parts                        |
|----------------------|------------------------------------|
| Rarely               | System prompt, examples            |
| When you ask         | Read-only files, repo map          |
| Every turn           | History, added files, your message |

**Stable first, volatile last. The order is a design decision.**

# Prefix reuse

---

Opening unchanged

The server skips re-reading it

---

One token different

Everything after it is redone

---

- Switching between ask and code
- Adding or dropping a file
- Naming a file in your message

**So the cheapest thing to change is the last thing you send.**

THE REPLY

# Edit-shaped replies

```
path/to/file.py
<<<<<<< SEARCH
the lines exactly as they appear on disk
=====
the lines that replace them
>>>>>>> REPLACE
```

**One block, one file. Nothing forces the set to be complete.**

# An action through a text channel

| The model returns | The harness needs    |
|-------------------|----------------------|
| Prose             | A file path          |
| Prose             | An exact region      |
| Prose             | The replacement text |

Search blocks, tool-call JSON, tagged output. One problem.

**Your parser is what turns a reply into an edit, or refuses to.**

# Errors written for the model

Aider answers the model, not you.

- The failed block, quoted back
- Lines from the file that nearly match
- A note if the work already exists

**An error message is a prompt.**

# How a file update arrives

Each added file is read from disk and sent whole, every turn.

|            |                            |
|------------|----------------------------|
| Never sent | A diff of what changed     |
| Never kept | The previous copy          |
| Also sent  | A line saying it committed |

**The model is never told what changed, only what is true now.**

STALE GROUND

# What it remembers

The file block is rebuilt. The model's own replies are not.

**It can see the lines it chose to change, and nothing else.**

# The silent undo

## After an undo

|             |                                  |
|-------------|----------------------------------|
| The file    | Quietly back to the old text     |
| The model   | Told nothing at all              |
| The history | Still holds the edit it proposed |

Its best evidence says the change was made.

**This is the reason your undo has to be owned and current-session.**

# Hackathon 1

ONE FEATURE ON YOUR OWN BUILD, IN ONE EVENING

THURSDAY SEPTEMBER 24

# The format

- Two hours, your own project
- Prompt revealed in the room
- Submit before you leave
- Graded separately from the build

# The hackathon workspace

- Browser VS Code, already set up
- Aider pointed at the course endpoint
- Your repository already cloned
- A Python reference pack, offline

# The pair-programmer

WHAT STAGE 1 ASKS FOR

# A harness from scratch

Everything in the last hour, written by you.

|           |                                |
|-----------|--------------------------------|
| It holds  | A set of files you chose       |
| It asks   | A model endpoint you configure |
| It writes | Only after you approve         |

Aider without Aider. Same job, your code.

Due **Tuesday October 6, 11:59 PM.**

# Specification before implementation

You write the requirements before you write the code, and the commit history has to show it.

|                 |                                    |
|-----------------|------------------------------------|
| System design   | Once, before implementation        |
| Increment specs | Slices you choose, each spec first |

No specification is supplied.

# The design artifacts

| Artifact          | Covers                               |
|-------------------|--------------------------------------|
| Your requirements | Config, run contract, seven features |
| Increment specs   | Ordered, testable, context scoped    |
| Three diagrams    | Components, sequence, edit lifecycle |

# Specification and design

**50 of the 100 build points.**

| Criterion                            | Points    |
|--------------------------------------|-----------|
| Requirements and acceptance criteria | 10        |
| Architecture and interface design    | 10        |
| Increment specs                      | 10        |
| The three diagrams                   | 4 / 3 / 3 |
| Specs before code                    | 5         |
| What changed and why                 | 5         |

# Implementation and tests

**50 of the 100 build points.**

| Criterion                         | Points |
|-----------------------------------|--------|
| Functional correctness            | 25     |
| Custom test cases                 | 9      |
| Failure-path tests                | 8      |
| End-to-end runs and live evidence | 5      |
| Operating instructions            | 3      |

`RUBRIC.md` carries every row.

WHAT YOU KEEP

# The evidence trail

|                |                                           |
|----------------|-------------------------------------------|
| Commits        | Specs before the code they describe       |
| Sessions       | Which context, which model, what happened |
| Reconciliation | Where the design changed, and why         |

# Model rules for specifications

- Drafting, critique, diagrams, code, tests
- 9B recommended, 4B permitted
- Secondary models recorded too
- Any compatible host allowed

**A different provider is not a different model.**

# Why this tool

THE TOOL YOU HAVE BEEN DRIVING, AND WHY IT WAS THIS ONE

# The visible joins

| The decision                | Where you saw it            |
|-----------------------------|-----------------------------|
| Which files the model sees  | You typed <code>/add</code> |
| Which model answers         | One line of config          |
| What a reply must look like | The block format            |
| What changed on disk        | The diff, then the commit   |

**A tool that decides for you hides what you came to learn.**

# What control costs

| What it gives you          | What it costs you           |
|----------------------------|-----------------------------|
| You choose the context     | You have to choose it       |
| Every decision is visible  | Nothing is done for you     |
| One loop you can follow    | It cannot plan ahead        |
| Nothing is written unasked | It forgets between sessions |

**Every row is one decision, read from both sides.**

THE SAME SHAPE, MORE MACHINERY

# Who finds the context

|                       |                          |
|-----------------------|--------------------------|
| You name the files    | Aider, when you add them |
| A heuristic guesses   | Aider, with the repo map |
| The model asks for it | The tools you meet next  |

**You are about to build the first two rows yourselves.**

# The part you own

|              |                                        |
|--------------|----------------------------------------|
| The model    | A name and a URL                       |
| The endpoint | Course server, your laptop, a provider |
| The harness  | Every decision in this deck            |

**The model is a dependency. The harness is the product.**

EVERY TOOL YOU WILL MEET

# The same loop, more machinery

- Assemble, answer, parse, apply, repair, commit
- Every tool you will meet runs it
- What differs is who takes each step

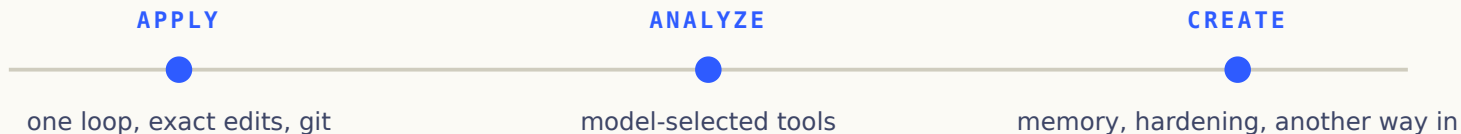
**One question is still open: who picks the next action?**

# What comes next

ONE REPOSITORY, GROWN UNTIL DECEMBER

THROUGH DECEMBER

# One repository to December



**Nothing is thrown away. Each phase grows the last.**

BY DECEMBER

# The harness by December

You will have built the kind of tool this lab took apart: a coding agent in the shape of Claude Code or OpenClaw, running against a model you configure, grown from the pair-programmer you start this week.

**Today you drive one. In December you will have written one.**

# Office hours

THE REST OF THE SESSION IS YOURS

# Questions?